



1. Problem Title & Link

Title: LeetCode 901 : Online Stock Span

Link: [LeetCode 901 - Online Stock Span](#)

2. Problem Statement (Short Summary)

Design a class:

StockSpanner

that collects daily stock prices and returns the stock span.

Stock Span:

For today's price, the span is the number of consecutive days (including today) where:

$\text{previousPrice} \leq \text{currentPrice}$

In simple terms:

Find how many previous days had stock prices less than or equal to today's price.

3. Examples (Input → Output)

Example 1

Input:

```
["StockSpanner",  
 "next",  
 "next",  
 "next",  
 "next",  
 "next",  
 "next",  
 "next"]
```

```
[[],  
 [100],  
 [80],  
 [60],  
 [70],  
 [60],  
 [75],  
 [85]]
```

Output:

```
[null,1,1,1,2,1,4,6]
```

Explanation:



Price	Span
100	1
80	1
60	1
70	2
60	1
75	4
85	6

4. Constraints

$1 \leq \text{price} \leq 10^5$

At most 10^4 calls to next()

Special restrictions:

- Need efficient processing
- Cannot scan previous prices repeatedly

5. Core Concept (Pattern / Topic)

Monotonic Stack

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every new price:

1. Traverse backward
2. Count consecutive smaller/equal prices

Time:

$O(n)$

per operation

Worst case:

$O(n^2)$

Too slow.

Optimized Thinking

Observation:

If a previous price is smaller than the current price:

it can never affect future comparisons

Maintain stack:



(price, span)

Rules:

1. Remove all smaller/equal prices
2. Add their spans
3. Push current value with total span

The stack always remains:

strictly decreasing

7. Visual / Intuition Diagram (ASCII Diagram)

Input sequence:

100,80,60,70,60,75,85

Stack evolution:

100

100

80

100

80

60

70 arrives

Remove 60

100

80

70(2)

75 arrives

Remove 70

Remove 60

100

80

75(4)



85 arrives

Remove 75

Remove 80

100

85(6)

8. Pseudocode (Language Independent)

stack=[]

function next(price):

 span=1

 while stack not empty
 and top.price<=price:

 span += top.span

 pop()

 push(price,span)

 return span

9. Code Implementation

Python

class StockSpanner:

 def __init__(self):

 self.stack=[]

 def next(self, price):



```
span=1

while (self.stack and
       self.stack[-1][0]
       <=price):

    span += (
        self.stack.pop()[1]
    )

self.stack.append(
    (price,span)
)

return span
```

Java

```
class StockSpanner {

    Stack<int[]> stack;

    public StockSpanner() {

        stack=new Stack<>();
    }

    public int next(int price) {

        int span=1;

        while(!stack.isEmpty()
              &&
              stack.peek()[0]
              <=price){

            span+=
```



```
        stack.pop()[1];
    }

    stack.push(
        new int[]{
            price,
            span
        });

    return span;
}
}
```

10. Time & Space Complexity

Time Complexity:

$O(1)$

Amortized

Reason:

Each element is pushed and popped only once.

Worst single operation:

$O(n)$

But total operations across all calls remain linear.

Space Complexity:

$O(n)$

Reason:

Stack stores stock prices.

11. Common Mistakes / Edge Cases

1. Using:

<

instead of:

<=

Problem requires:

smaller OR equal

2. Forgetting to accumulate previous span



Wrong:

span++

Correct:

span += previousSpan

3. Storing only prices

Need:

(price, span)

4. Missing increasing sequence:

10,20,30,40

12. Detailed Dry Run (Step-by-Step Table)

Input:

100,80,60,70,60,75,85

Price	Stack Before	Action	Span
100	[]	Push	1
80	100	Push	1
60	10,80	Push	1
70	10,80,60	Pop 60	2
60	10,80,70	Push	1
75	10,80,70,60	Pop 60,70	4
85	10,80,75	Pop 75,80	6

Output:

1,1,1,2,1,4,6

13. Common Use Cases (Real-Life / Interview)

Stock market analysis

Trend detection

Time-series processing

Monitoring systems

14. Common Traps (Important!)

1. Using brute force scanning
2. Missing equal-price condition



3. Forgetting span accumulation
4. Not maintaining decreasing stack

15. Builds To (Related LeetCode Problems)

Progression:

- LC 901 → Online Stock Span
- LC 739 → Daily Temperatures
- LC 496 → Next Greater Element I
- LC 503 → Next Greater Element II
- LC 84 → Largest Rectangle in Histogram

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Backward Scan	$O(n^2)$	$O(1)$	Too slow
Stack storing prices only	$O(n)$	$O(n)$	Extra processing
Monotonic Stack + Span	$O(1)$ amortized	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)

The stack keeps prices in decreasing order. Smaller or equal prices become irrelevant once a larger price arrives, so they are removed and their spans are merged into the current span.

18. Variations / Follow-Up Questions

1. Find previous greater element
2. Find next greater element
3. Support stock price updates
4. Return maximum span till now
5. Solve for temperature trend analysis
- 6.